Lecture 10:

Convolutional Neural Networks (CNN) – Part II

Markus Hohle

University California, Berkeley

Machine Learning Algorithms

MSSE 277B, 3 Units

Course Map

Lecture 1: Course Overview and Introduction to Machine Learning

classic ML tools & algorithms

Lecture 2: Bayesian Methods in Machine Learning

Lecture 3: Dimensionality Reduction: Principal Component Analysis

Lecture 4: Linear and Non-linear Regression and Classification

Lecture 5: Unsupervised Learning: K-Means, GMM, Trees

Lecture 6: Adaptive Learning and Gradient Descent Optimization Algorithms

ANNs/AI/Deep Learning

Lecture 7: Introduction to Artificial Neural Networks - The Perceptron

Lecture 8: Introduction to Artificial Neural Networks - Building Multiple Dense Layers

Lecture 9: Convolutional Neural Networks (CNNs) - Part I

Lecture 10: CNNs - Part II

Lecture 11: Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTMs)

Lecture 12: Combining LSTMs and CNNs

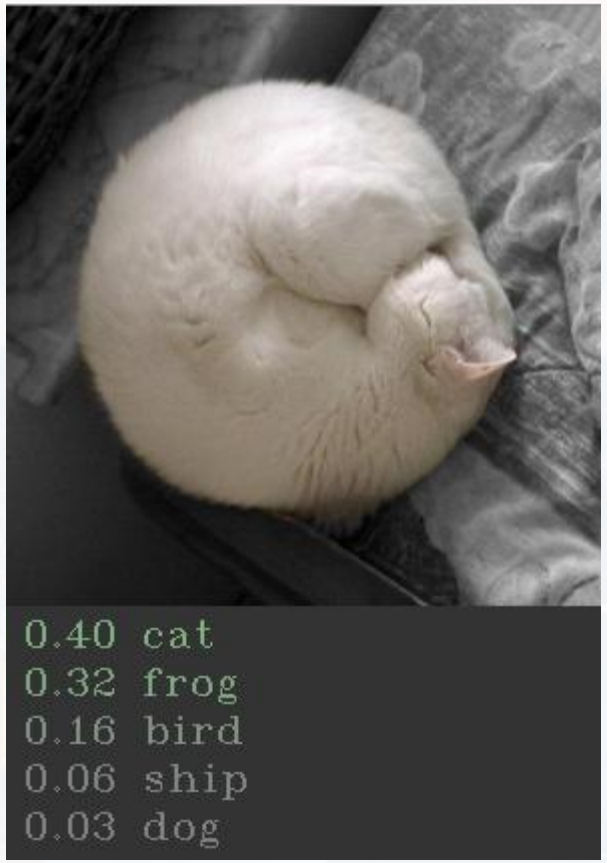
Lecture 13: Running Models on GPUs and Parallel Processing

Lecture 14: Project Presentations

Lecture 15: Transformer

Lecture 16: GNN

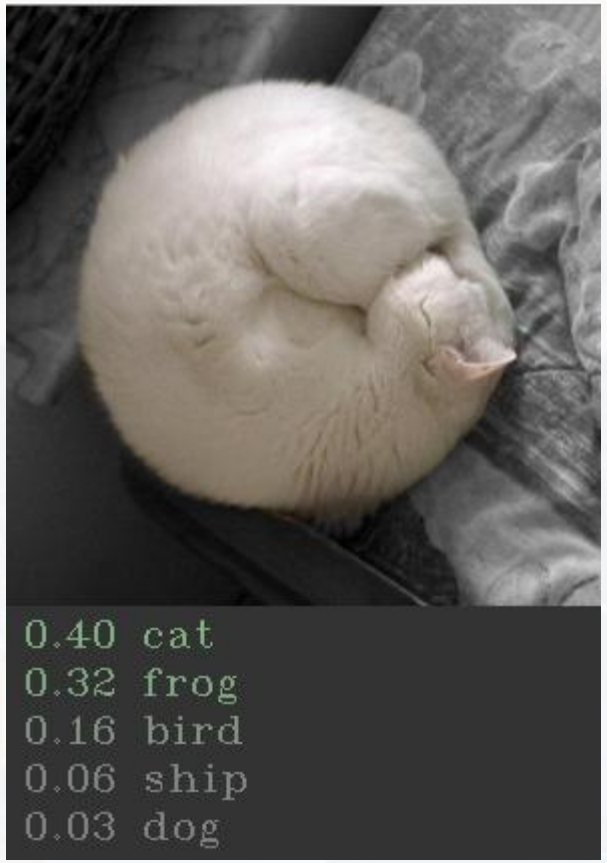
Machine Learning Algorithms:



Outline

- **Labeling Tools**
- **Calling a Pretrained CNN**
- **CNN Architectures**

Machine Learning Algorithms:



Outline

- **Labeling Tools**
- Calling a Pretrained CNN
- CNN Architectures

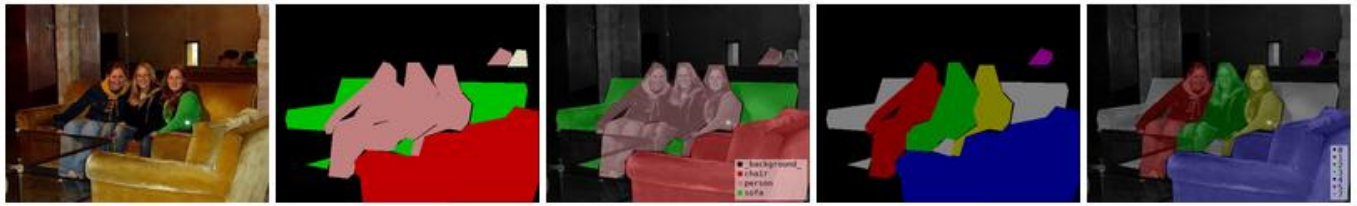
Convolutional Neural Networks:

Labeling

[“labelme”]

Description

Labelme is a graphical image annotation tool inspired by http://labelme.csail.mit.edu.
It is written in Python and uses Qt for its graphical interface.



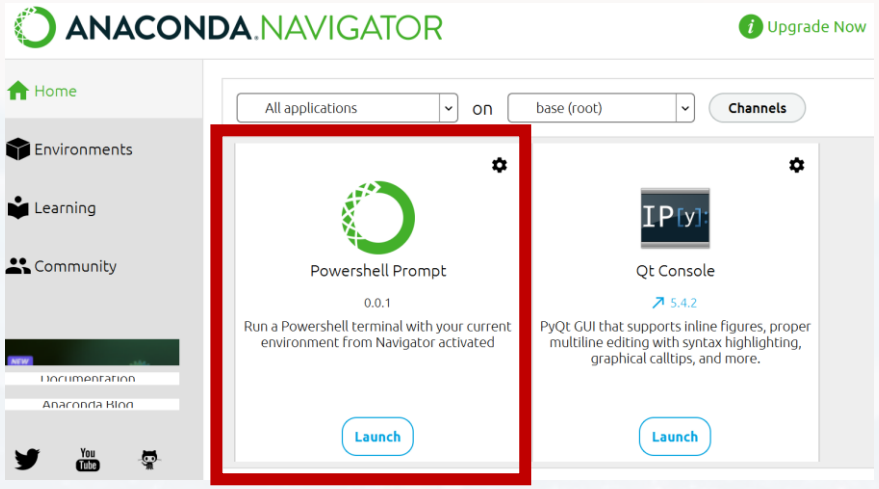
VOC dataset example of instance segmentation.

Other examples (semantic segmentation, bbox detection, and classification).

Various primitives (polygon, rectangle, circle, line, and point).

Convolutional Neural Networks:

Labeling




```
conda install labelme
```

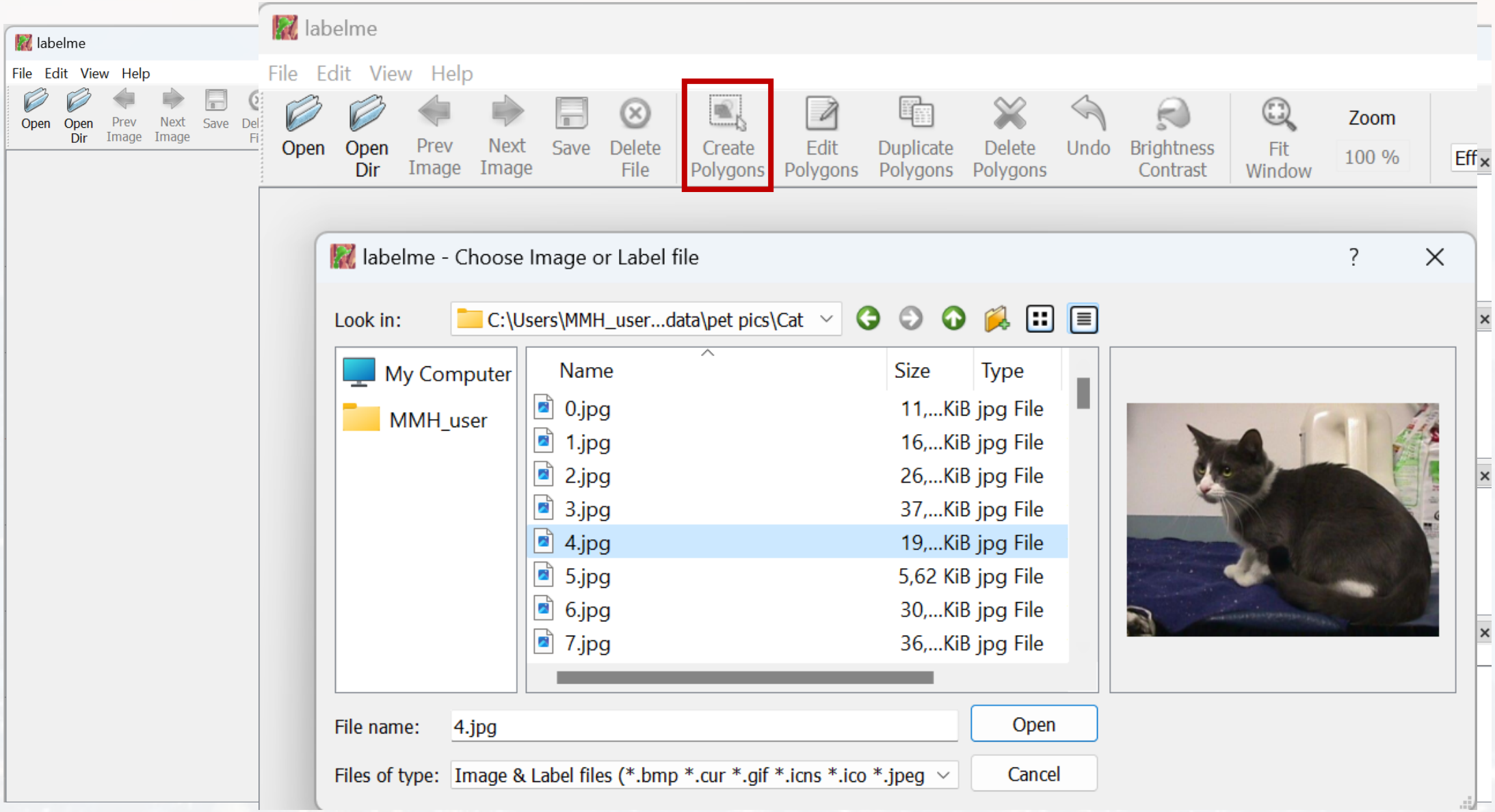
```
C:\WINDOWS\System32\Win
(base) PS C:\Users\MMH_user> conda create --name=labelme python=3
```

```
(base) PS C:\Users\MMH_user> conda activate labelme
```

```
(labelme) PS C:\Users\MMH_user> labelme
```

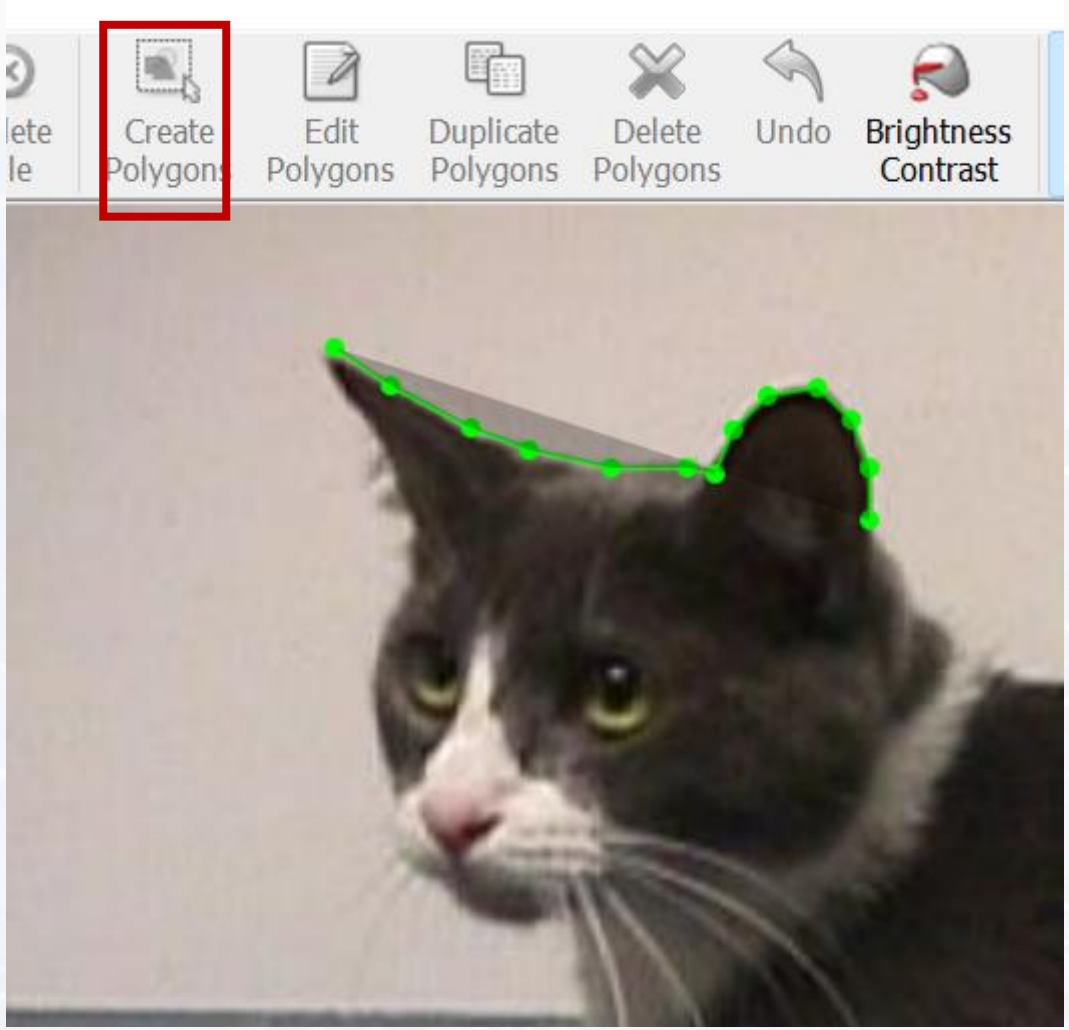
Convolutional Neural Networks:

Labeling

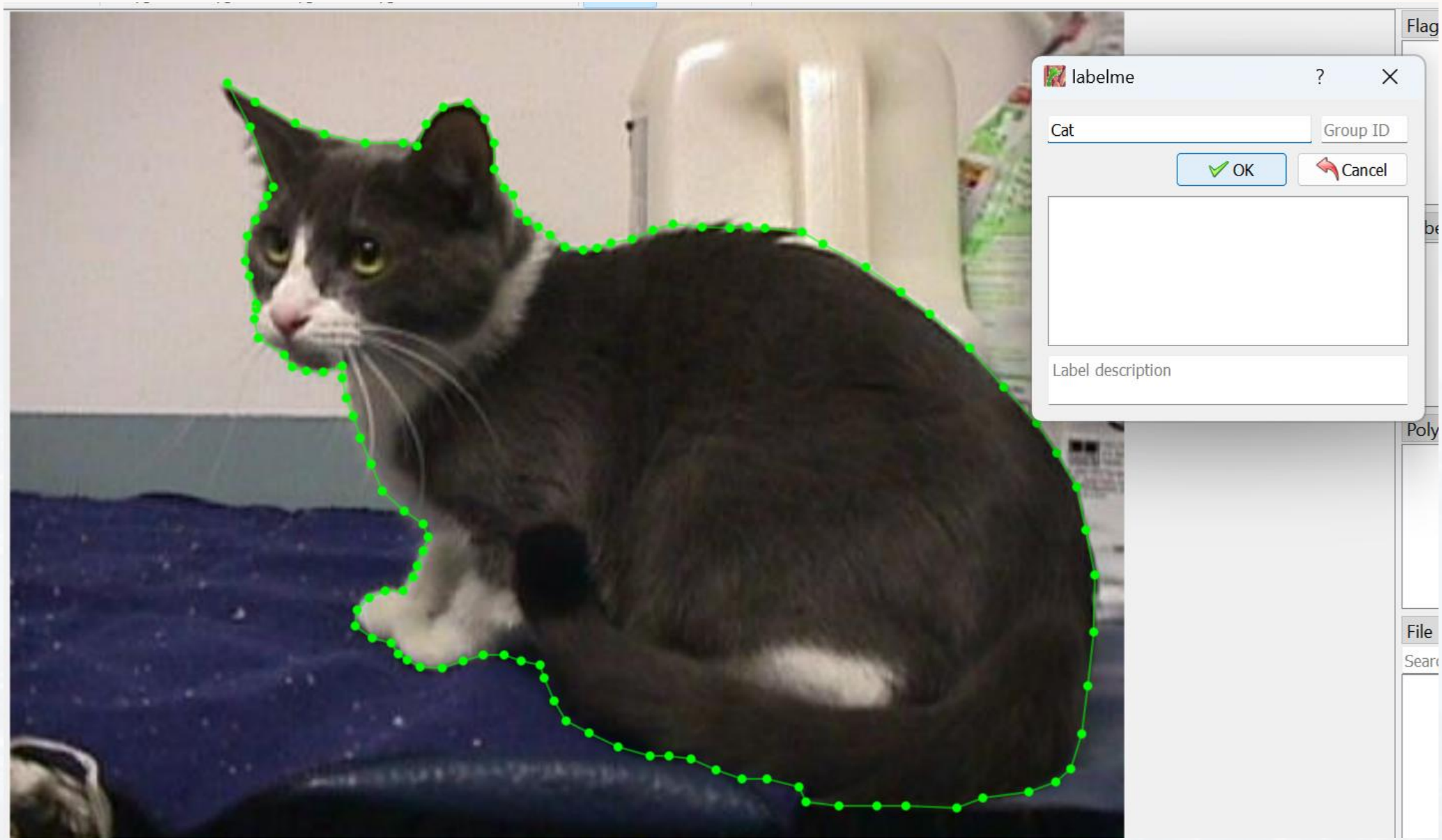


Convolutional Neural Networks:

Labeling



Convolutional Neural Networks: Labeling



Convolutional Neural Networks: Labeling

Convolutional Neural Networks:

image is saved as `.json`

run within the labelme prompt:

```
labelme_export_json .\Cat4label.json -o Cat4label_json
```

Convolutional Neural Networks:

Labeling

PNG File

img

label

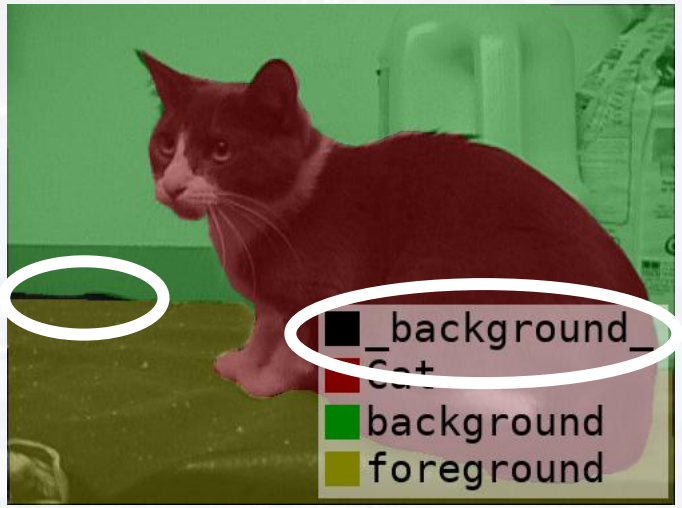
label_viz

Text Document

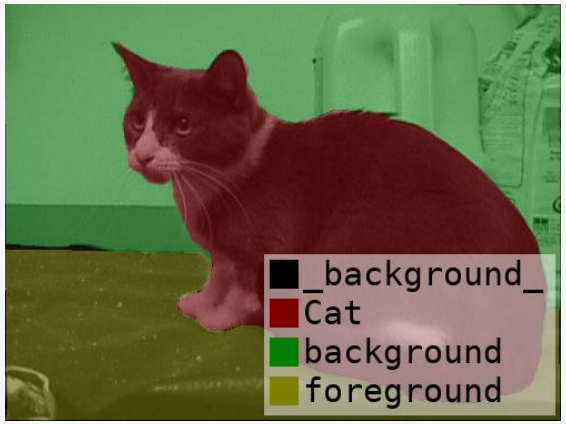
label_names

File Edit View

```
_background_
Cat
background
foreground
```



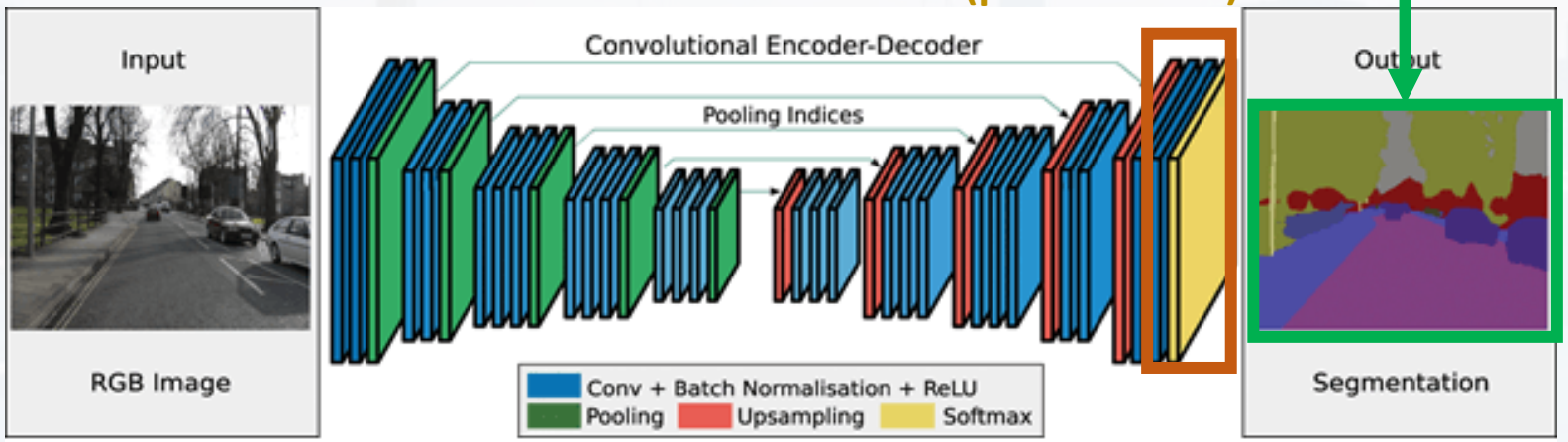
Convolutional Neural Networks:



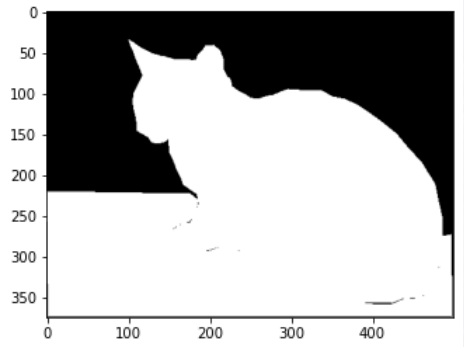

```
I = plt.imread('Cat/Cat4label_json/label.png')
```

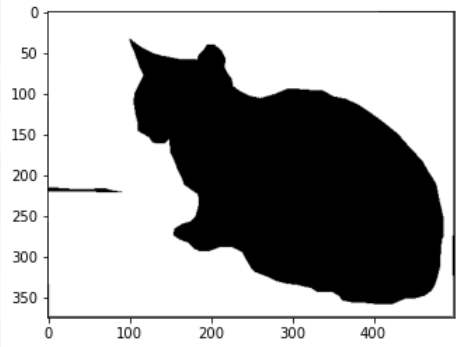
```
I.shape
(375, 500, 4)
```

softmax
(probabilities)



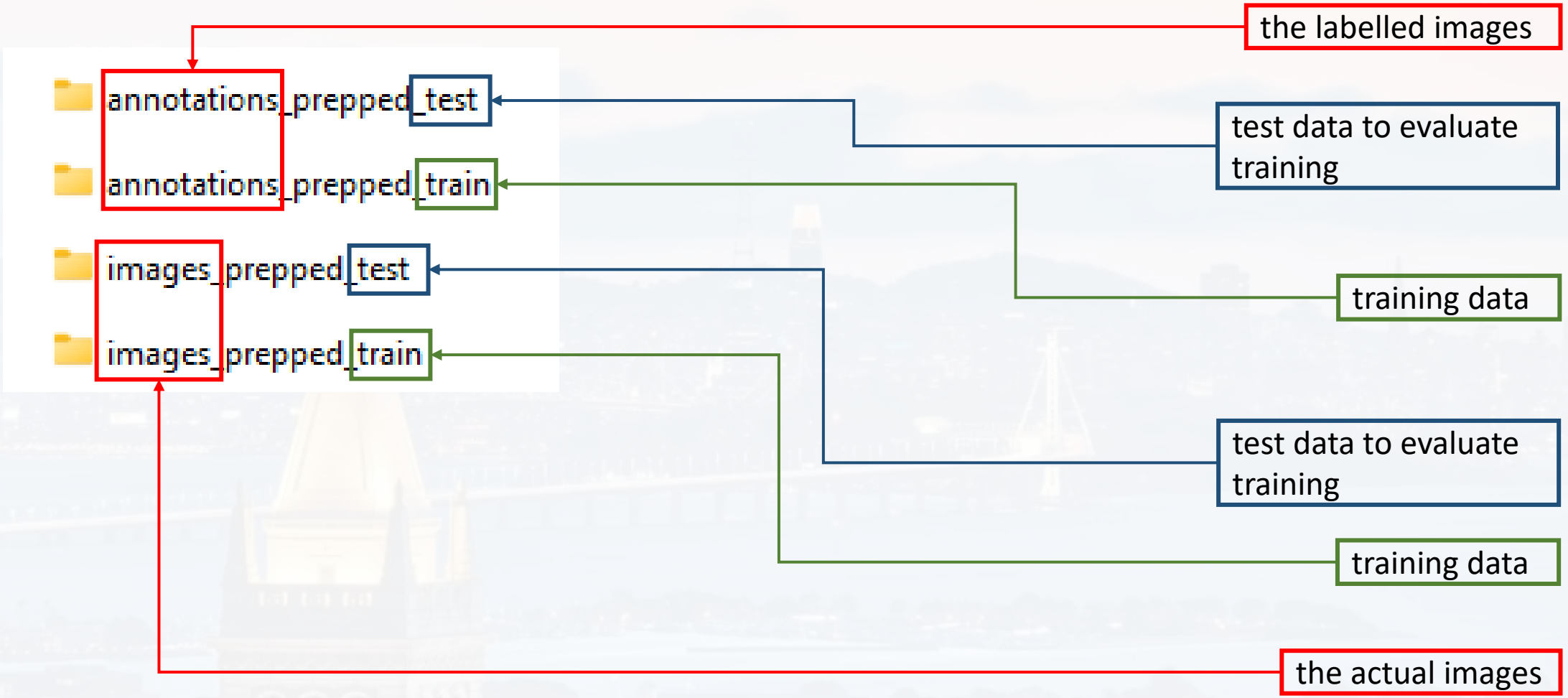
labels assigned during training vs actual labels





Convolutional Neural Networks:

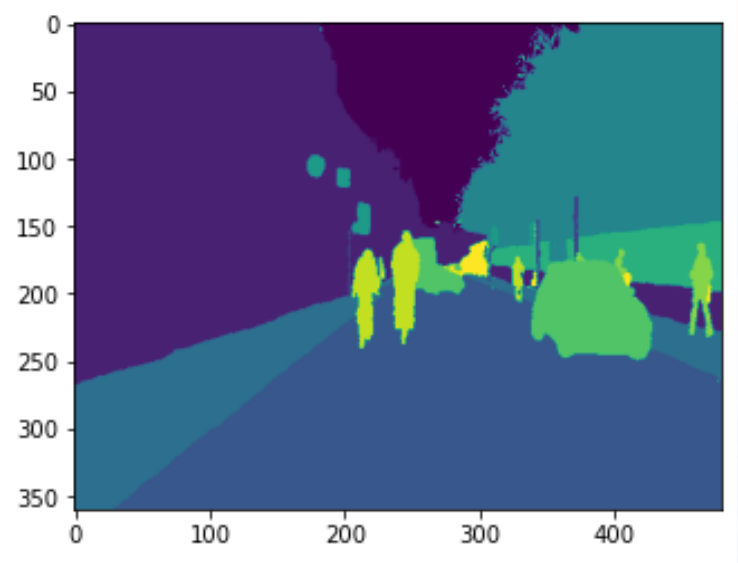
Labeling



Convolutional Neural Networks: Labeling

nice dataset

images_prepped_test

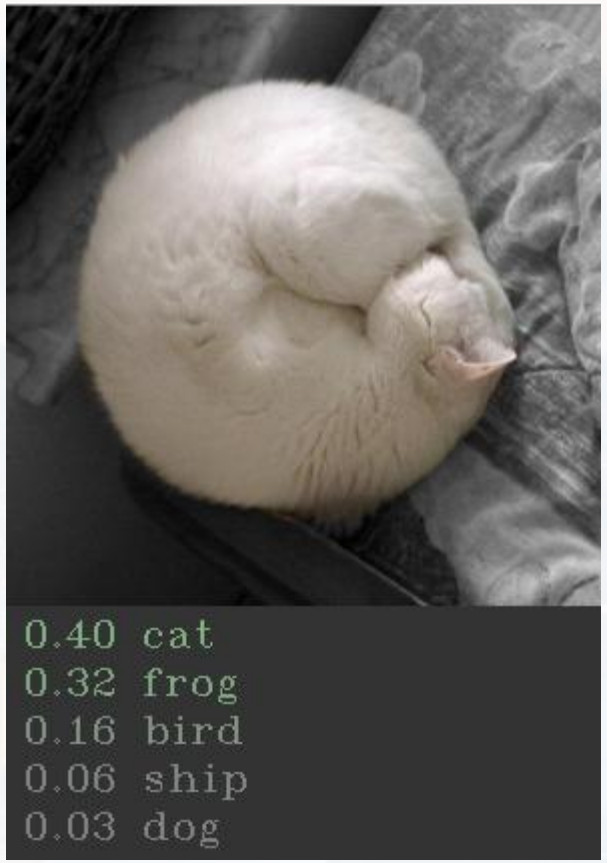


annotations_prepped_test

```
I = plt.imread('segmentation/pics/annotations_prepped_test/0016E5_07959.png')
```

```
I.shape
(360, 480)
```

Machine Learning Algorithms:



Outline

- Labeling Tools
- **Calling a Pretrained CNN**
- CNN Architectures

Convolutional Neural Networks: Segmentation

demonstrating realistic segmentation with reasonable results will take a few hours

→ check out my code **`SegmentMyImages.py`**

Convolutional Neural Networks: Segmentation

```
from keras_segmentation.models.unet import *
```

Type	Names
VGG	'vgg16' 'vgg19'
ResNet	'resnet18' 'resnet34' 'resnet50' 'resnet101' 'resnet152'
SE-ResNet	'seresnet18' 'seresnet34' 'seresnet50' 'seresnet101' 'seresnet152'
ResNeXt	'resnext50' 'resnext101'
SE-ResNeXt	'seresnext50' 'seresnext101'
SENet154	'senet154'
DenseNet	'densenet121' 'densenet169' 'densenet201'
Inception	'inceptionv3' 'inceptionresnetv2'
MobileNet	'mobilenet' 'mobilenetv2'
EfficientNet	'efficientnetb0' 'efficientnetb1' 'efficientnetb2' 'efficientnetb3' 'efficientnetb4' 'efficientnetb5' efficientnetb6' efficientnetb7'

```
model = unet(n_classes = n_classes,\
                         input_height = 416, input_width = 608)

model.train(
        train_images        = my_path + r"images_prepped_train//",
        train_annotations = my_path + r"annotations_prepped_train//",
        checkpoints_path   = my_path + r"checkpoints//",
        do_augment                  = True,
        gen_use_multiprocessing = True,
        auto_resume_checkpoint   = True,
        epochs = 5)
```

calling the specific network

saves current weights

Keras provides an augmentation routine

Convolutional Neural Networks:


```
model = vgg_unet(n_classes = n_classes,\
                         input_height = 416, input_width = 608)

model.train(
        train_images      = my_path + r"images_prepped_train//",
        train_annotations = my_path + r"annotations_prepped_train//",
        checkpoints_path  = my_path + r"checkpoints//",
        do_augment                  = True,
        gen_use_multiprocessing = True,
        auto_resume_checkpoint  = True,
        epochs = 5)
```

Keras provides an augmentation routine

Note: I always run my **own augmentation** routine, see e.g. `AugmentMyImages.py`

run:

```
S = SegmentMyImages()

S.Training()
```

Convolutional Neural Networks:


```
S = SegmentMyImages()

S.Training()
```

```
Dataset verified!
Epoch 1/5
512/512 [==============================] - ETA: 0s - loss: 4.1406 - accuracy: 0.0353     saved  ../data/segmentation
pics/checkpoints//.0
512/512 [==============================] - 3888s 8s/step - loss: 4.1406 - accuracy: 0.0353
Epoch 2/5
512/512 [==============================] - ETA: 0s - loss: 3.7805 - accuracy: 0.1636     saved  ../data/segmentation
pics/checkpoints//.1
512/512 [==============================] - 4133s 8s/step - loss: 3.7805 - accuracy: 0.1636
Epoch 3/5
512/512 [==============================] - ETA: 0s - loss: 3.4534 - accuracy: 0.3338     saved  ../data/segmentation
pics/checkpoints//.2
512/512 [==============================] - 4072s 8s/step - loss: 3.4534 - accuracy: 0.3338
Epoch 4/5
512/512 [==============================] - ETA: 0s - loss: 3.1926 - accuracy: 0.3980     saved  ../data/segmentation
pics/checkpoints//.3
512/512 [==============================] - 3363s 7s/step - loss: 3.1926 - accuracy: 0.3980
Epoch 5/5
512/512 [==============================] - ETA: 0s - loss: 3.0071 - accuracy: 0.4318   saved  ../data/segmentation
pics/checkpoints//.4
512/512 [==============================] - 3616s 7s/step - loss: 3.0071 - accuracy: 0.4318
```

Convolutional Neural Networks:


```
MyModel = S.TrainedModel
```

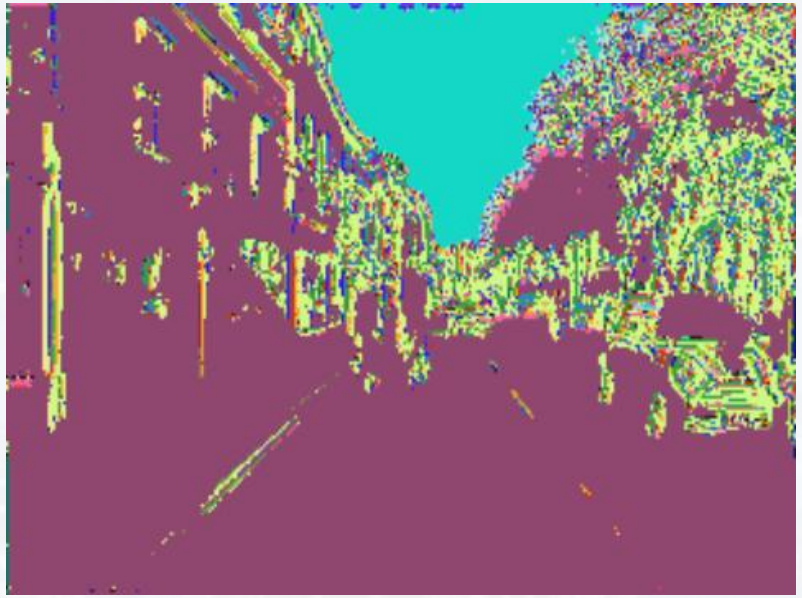
```
MyModel.summary()
```

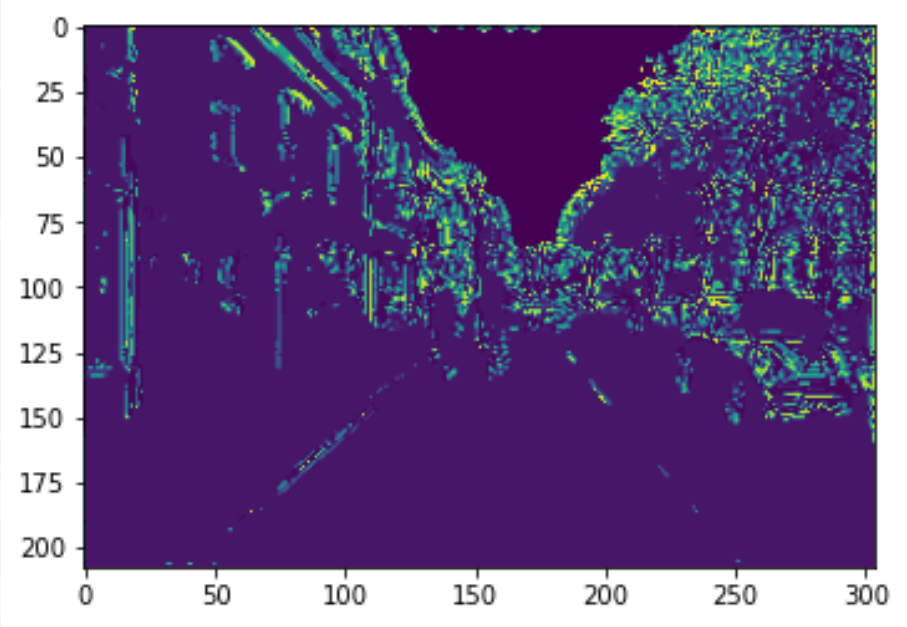
→ returns the structure of the CNN

applying the trained CNN to an image:

```
out = S.ApplyTrainedNetwork()
```

```
plt.imshow(out)
```





Convolutional Neural Networks:

recovering model from checkpoints:

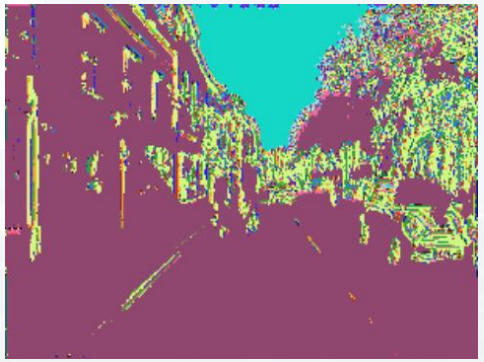
```
MyModel = S.TrainedModel
out     = S.ApplyTrainedNetwork()

S.RecoverFromCheckpoint()
```

applying the trained CNN to an image:

```
#loading untrained CNN
model = self.model

if not image_name:
    image_name = '0016E5_07965.png'
    
#calling input from checkpoints
latest = tf.train.latest_checkpoint(self.checkpoint_path)
model.load_weights(latest)
```

untrained model (just CNN itself)

transfer the saved weights to untrained network → now it starts from latest training state



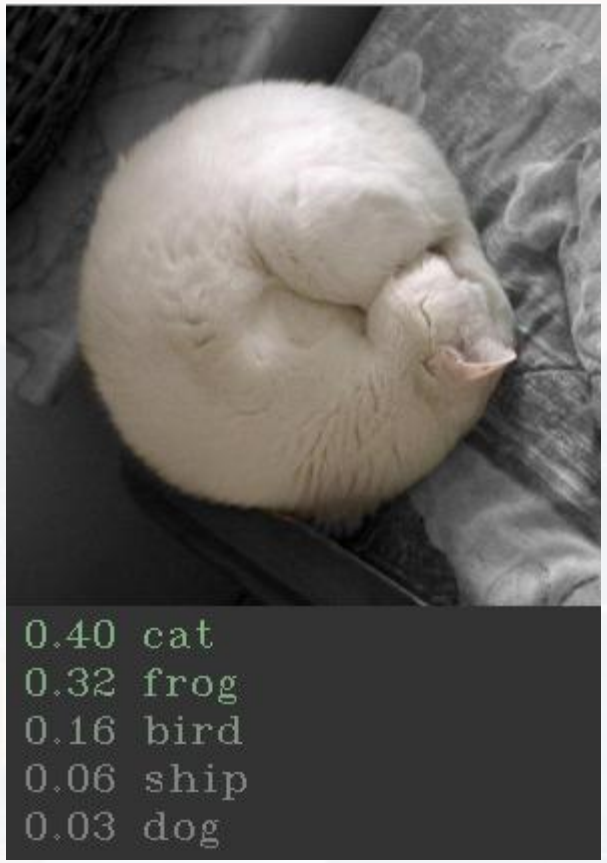
Convolutional Neural Networks:

visualizing weights:

→ see `model.layers`

nice example

Machine Learning Algorithms:



Outline

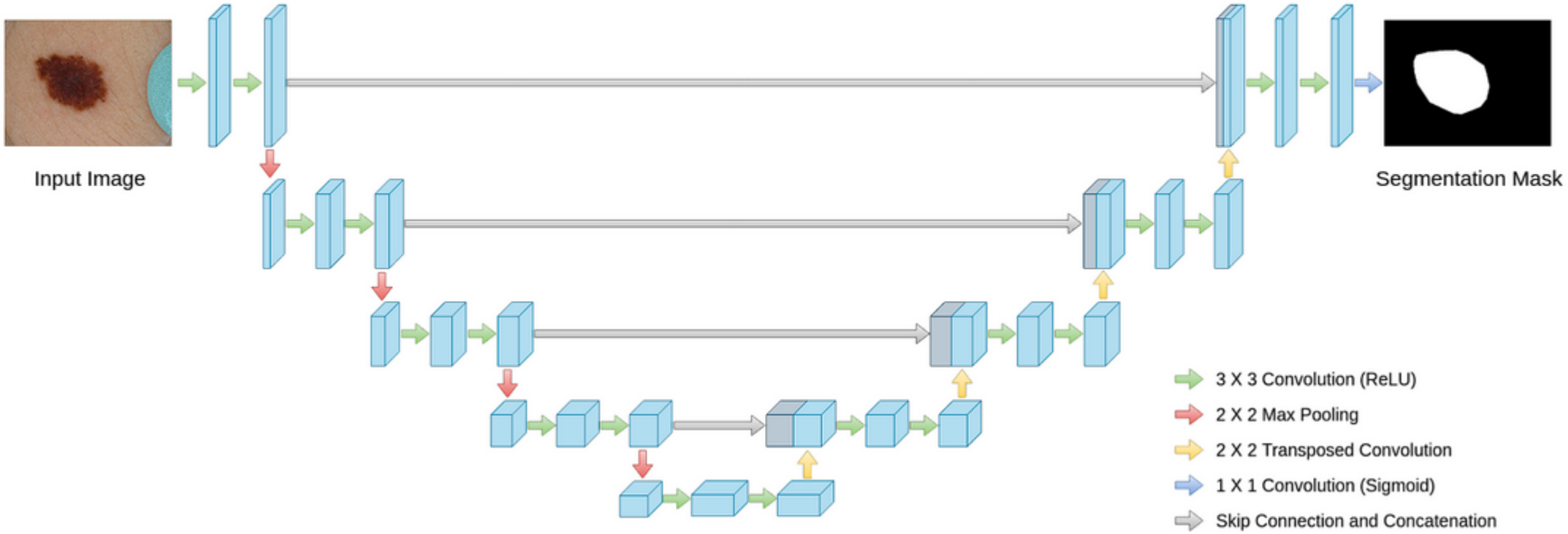
- Labeling Tools
- Calling a Pretrained CNN
- **CNN Architectures**

Convolutional Neural Networks:

Unet architecture

see `Unet architecture.ipynb`

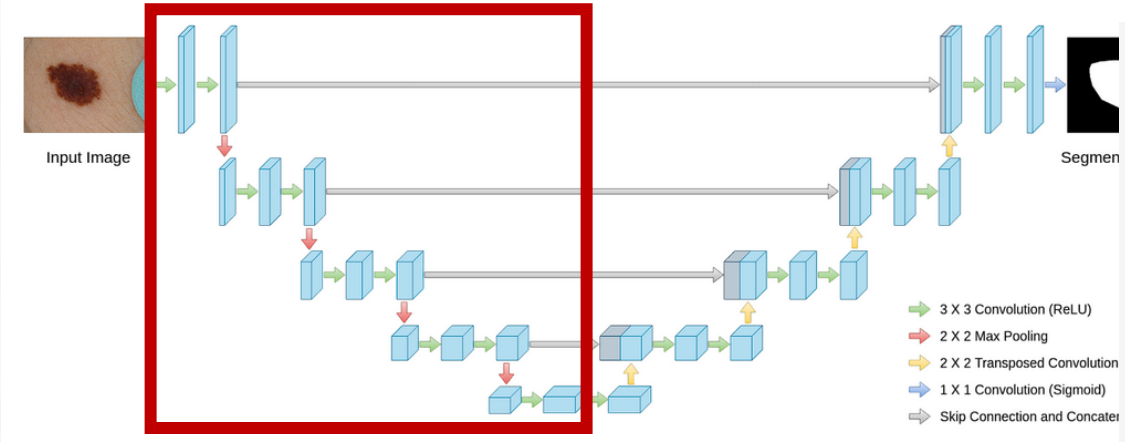
https://www.kaggle.com/code/abdallahwagih/brain-tumor-segmentation-unet-dice-coef-89-6/notebook



Convolutional Neural Networks: CNN Architectures

Unet architecture

https://www.kaggle.com/code/abdallahwagih/brain-tumor-segmentation-unet-dice-coef-89-6/notebook



see `Unet architecture.ipynb`

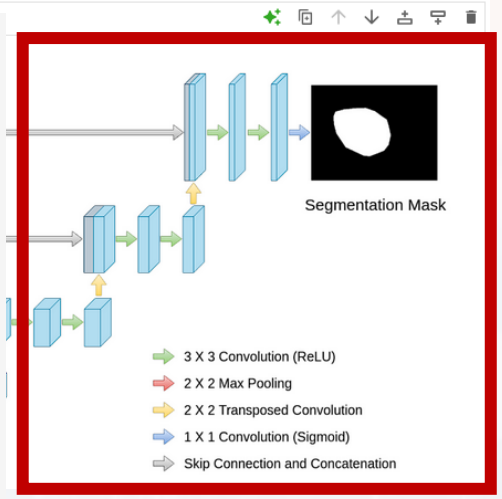
```python
def unet(input_size = (256, 256, 3)):
    inputs = Input(input_size)

    # First DownConvolution / Encoder Leg will begin, so start with Conv2D
    conv1 = Conv2D(filters = 64, kernel_size = (3, 3), padding = "same")(inputs)
    bn1   = Activation("relu")(conv1)
    conv1 = Conv2D(filters = 64, kernel_size = (3, 3), padding = "same")(bn1)
    bn1   = BatchNormalization(axis = 3)(conv1)
    bn1   = Activation("relu")(bn1)
    pool1 = MaxPooling2D(pool_size = (2, 2))(bn1)

    conv2 = Conv2D(filters = 128, kernel_size = (3, 3), padding = "same")(pool1)
    bn2   = Activation("relu")(conv2)
    conv2 = Conv2D(filters = 128, kernel_size = (3, 3), padding = "same")(bn2)
    bn2   = BatchNormalization(axis = 3)(conv2)
    bn2   = Activation("relu")(bn2)
    pool2 = MaxPooling2D(pool_size = (2, 2))(bn2)

    conv3 = Conv2D(filters = 256, kernel_size = (3, 3), padding = "same")(pool2)
    bn3   = Activation("relu")(conv3)
    conv3 = Conv2D(filters = 256, kernel_size = (3, 3), padding = "same")(bn3)
    bn3   = BatchNormalization(axis = 3)(conv3)
    bn3   = Activation("relu")(bn3)
    pool3 = MaxPooling2D(pool_size = (2, 2))(bn3)

    conv4 = Conv2D(filters = 512, kernel_size = (3, 3), padding = "same")(pool3)
    bn4   = Activation("relu")(conv4)
    conv4 = Conv2D(filters = 512, kernel_size = (3, 3), padding = "same")(bn4)
    bn4   = BatchNormalization(axis = 3)(conv4)
    bn4   = Activation("relu")(bn4)
    pool4 = MaxPooling2D(pool_size = (2, 2))(bn4)

    conv5 = Conv2D(filters = 1024, kernel_size = (3, 3), padding = "same")(pool4)
    bn5   = Activation("relu")(conv5)
    conv5 = Conv2D(filters = 1024, kernel_size = (3, 3), padding = "same")(bn5)
    bn5   = BatchNormalization(axis = 3)(conv5)
    bn5   = Activation("relu")(bn5)
```

Convolutional Neural Networks:

Unet architecture

see `Unet architecture.ipynb`

```
up6   = concatenate([Conv2DTranspose(512, kernel_size = (2, 2), strides = (2, 2), padding = "same")(bn5), conv4], axis = 3)
conv6 = Conv2D(filters = 512, kernel_size = (3, 3), padding = "same")(up6)
bn6   = Activation("relu")(conv6)
conv6 = Conv2D(filters = 512, kernel_size = (3, 3), padding = "same")(bn6)
bn6   = BatchNormalization(axis = 3)(conv6)
bn6   = Activation("relu")(bn6)

up7   = concatenate([Conv2DTranspose(256, kernel_size = (2, 2), strides = (2, 2), padding = "same")(bn6), conv3], axis = 3)
conv7 = Conv2D(filters = 256, kernel_size = (3, 3), padding = "same")(up7)
bn7   = Activation("relu")(conv7)
conv7 = Conv2D(filters = 256, kernel_size = (3, 3), padding = "same")(bn7)
bn7   = BatchNormalization(axis = 3)(conv7)
bn7   = Activation("relu")(bn7)

up8   = concatenate([Conv2DTranspose(128, kernel_size = (2, 2), strides = (2, 2), padding = "same")(bn7), conv2], axis = 3)
conv8 = Conv2D(filters = 128, kernel_size = (3, 3), padding = "same")(up8)
bn8   = Activation("relu")(conv8)
conv8 = Conv2D(filters = 128, kernel_size = (3, 3), padding = "same")(bn8)
bn8   = BatchNormalization(axis = 3)(conv8)
bn8   = Activation("relu")(bn8)

up9   = concatenate([Conv2DTranspose(64, kernel_size = (2, 2), strides = (2, 2), padding = "same")(bn8), conv1], axis = 3)
conv9 = Conv2D(filters = 64, kernel_size = (3, 3), padding = "same")(up9)
bn9   = Activation("relu")(conv9)
conv9 = Conv2D(filters = 64, kernel_size = (3, 3), padding = "same")(bn9)
bn9   = BatchNormalization(axis = 3)(conv9)
bn9   = Activation("relu")(bn9)

conv10 = Conv2D(filters = 1, kernel_size = (1, 1), activation = "sigmoid")(bn9)

return Model(inputs = [inputs], outputs = [conv10])
```

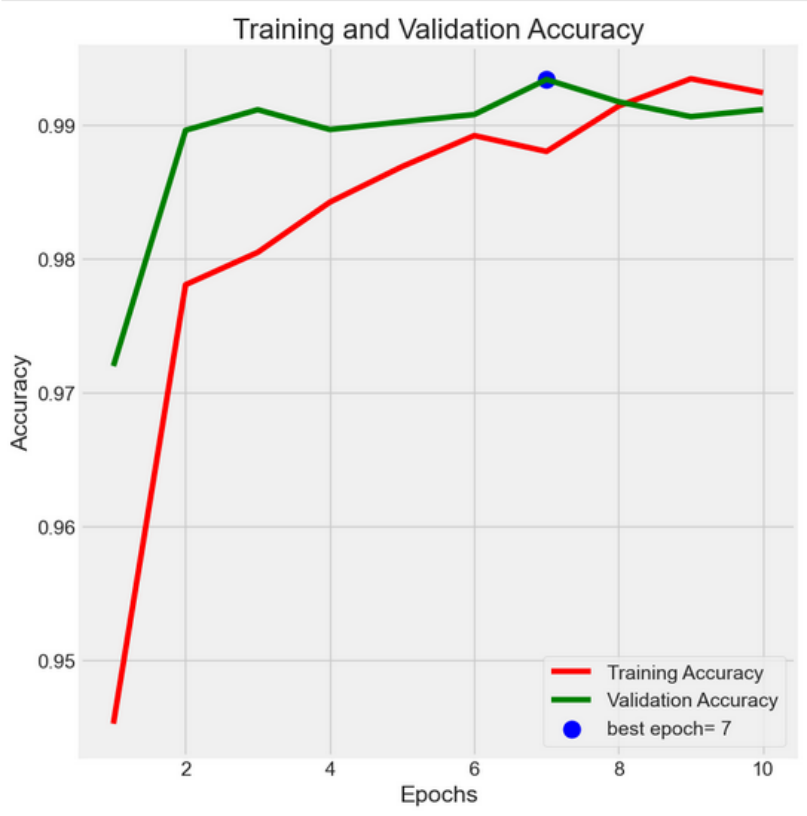


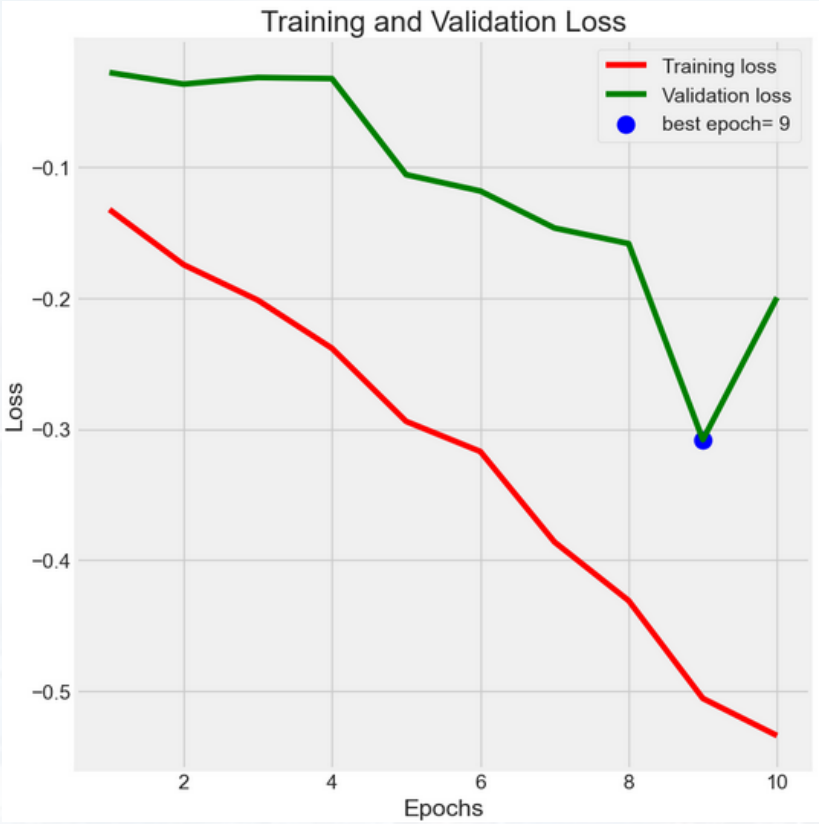
Convolutional Neural Networks:

Unet architecture

see `Unet architecture.ipynb`

```
---------------------------------------------------
Total params: 31043521 (118.42 MB)
Trainable params: 31037633 (118.40 MB)
Non-trainable params: 5888 (23.00 KB)
```





Convolutional Neural Networks:

Unet architecture

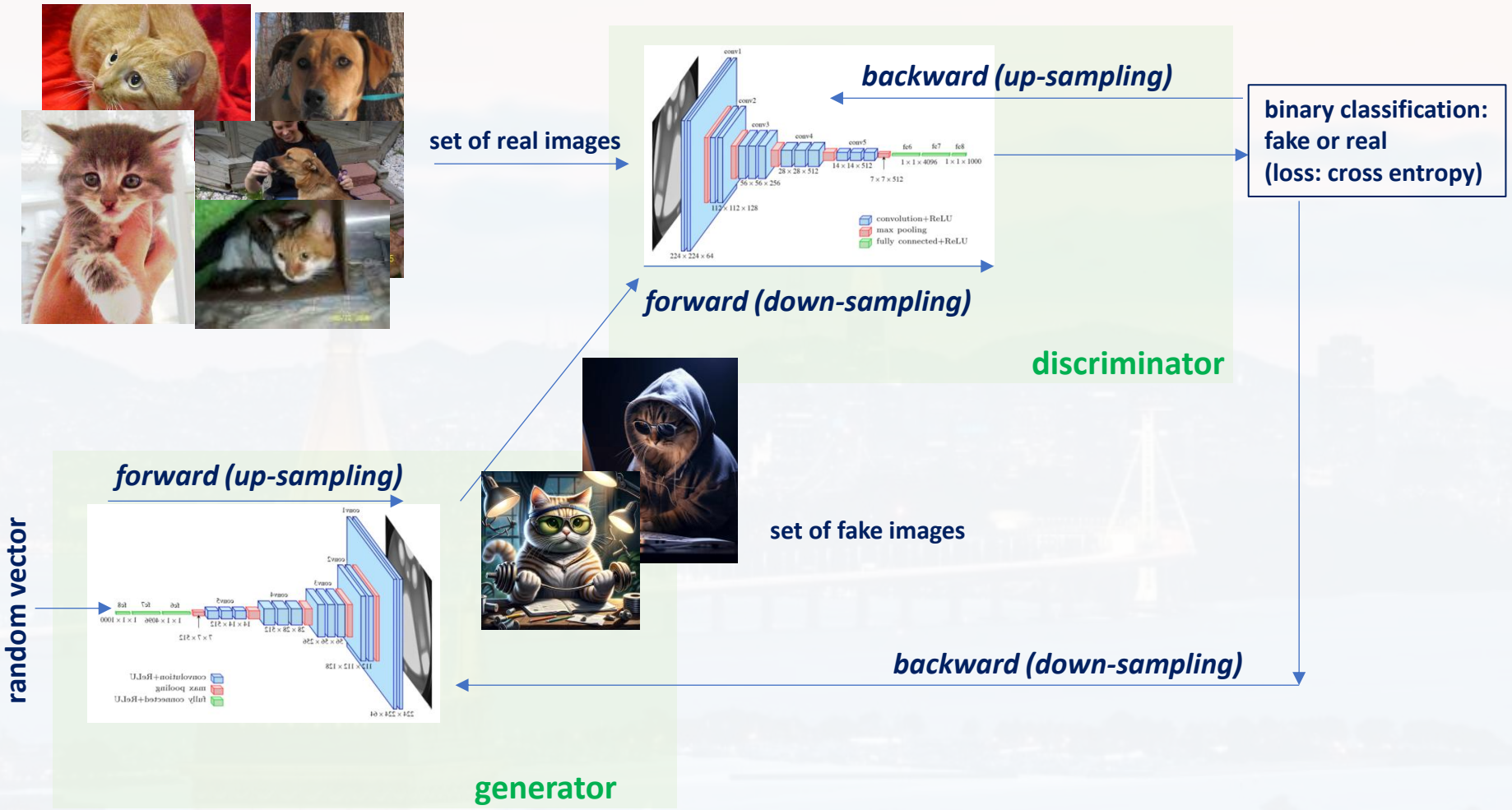
see `Unet architecture.ipynb`

```
---------------------------------------------------------------
Total params: 31043521 (118.42 MB)
Trainable params: 31037633 (118.40 MB)
Non-trainable params: 5888 (23.00 KB)
```

3,000 training images, 400 validation images, 10 epochs, 10hrs (Lenovo T14)

Convolutional Neural Networks:

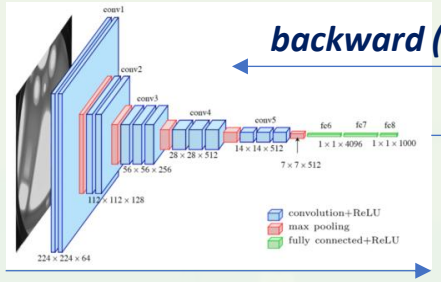
Generative Adversarial Network (GAN)



Convolutional Neural Networks:

Generative Adversarial Network (GAN)

set of real images



backward (up-sampling)

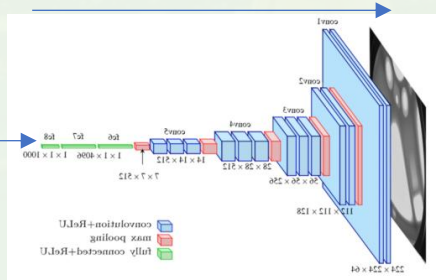
forward (down-sampling)

binary classification:
fake or real
(loss: cross entropy)

set of fake images

random vector

forward (up-sampling)



backward

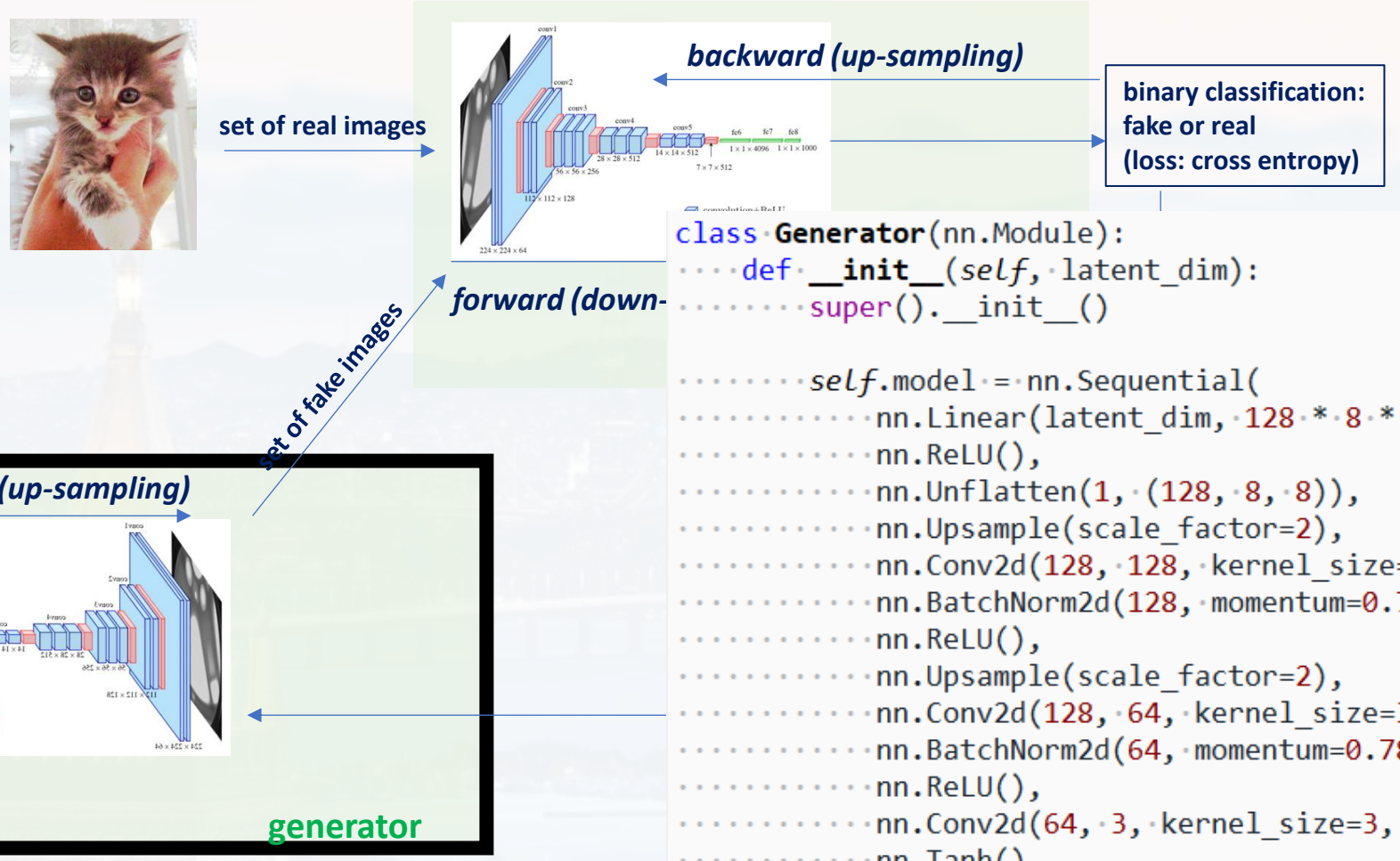
generator

```
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()

        self.model = nn.Sequential(
        nn.Conv2d(3, 32, kernel_size=3, stride=2, padding=1),
        nn.LeakyReLU(0.2),
        nn.Dropout(0.25),
        nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1),
        nn.ZeroPad2d((0, 1, 0, 1)),
        nn.BatchNorm2d(64, momentum=0.82),
        nn.LeakyReLU(0.25),
        nn.Dropout(0.25),
        nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1),
        nn.BatchNorm2d(128, momentum=0.82),
        nn.LeakyReLU(0.2),
        nn.Dropout(0.25),
        nn.Conv2d(128, 256, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(256, momentum=0.8),
        nn.LeakyReLU(0.25),
        nn.Dropout(0.25),
        nn.AdaptiveAvgPool2d((1, 1)),
        nn.Sigmoid()
```

standard classification CNN

Convolutional Neural Networks:

Generative Adversarial Network (GAN)




```python
class Generator(nn.Module):
    def __init__(self, latent_dim):
        super().__init__()

        self.model = nn.Sequential(
            nn.Linear(latent_dim, 128 * 8 * 8),
            nn.ReLU(),
            nn.Unflatten(1, (128, 8, 8)),
            nn.Upsample(scale_factor=2),
            nn.Conv2d(128, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128, momentum=0.78),
            nn.ReLU(),
            nn.Upsample(scale_factor=2),
            nn.Conv2d(128, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64, momentum=0.78),
            nn.ReLU(),
            nn.Conv2d(64, 3, kernel_size=3, padding=1),
            nn.Tanh()
        )

    def forward(self, z):
        img = self.model(z)
        return img
```

up-sampling: like in standard segmentation CNN

Generative Adversarial Network (GAN)

x: real image
z: latent vector (random numbers)

$p_D(x)$: probability that discriminator *D* classifies *x* as real image
$G(z)$: fake image generated from generator *G*

$\mathcal{L}_D$: loss function of discriminator *D*
$\mathcal{L}_G$: loss function of generator *G*

(mean) binary cross entropy (BCE): $\mathcal{L}_D = -\frac{1}{N}\sum_{i=1}^{N}\log[p_D(x_i)] + \log[1 - p_D(G(z_i))]$

over all *N* images $\mathcal{L}_G = \frac{1}{N}\sum_{i=1}^{N}\log[1 - p_D(G(z_i))]$

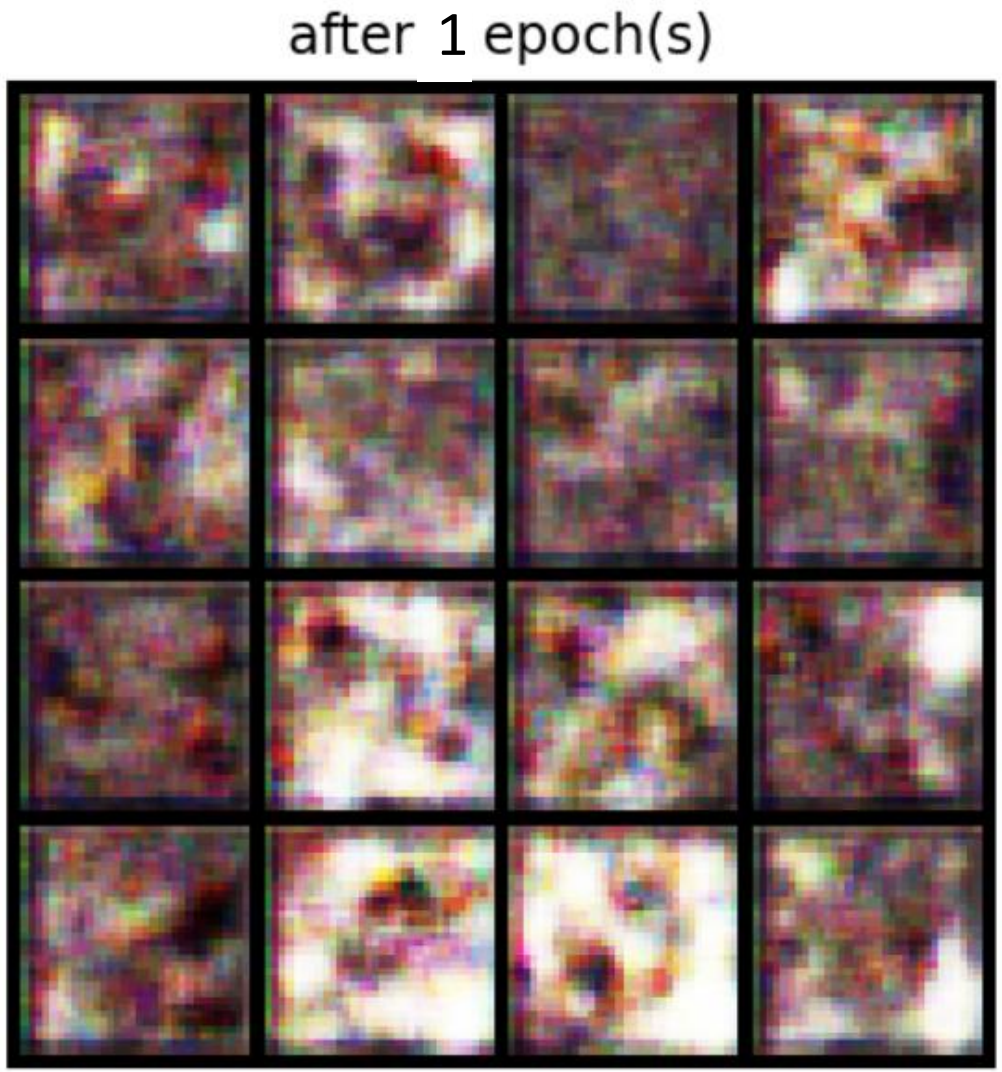
ideal case for *D*: $\mathcal{L}_D = -\log[1] - \log[1 - 0] = 0$
worst case for *D*: $\mathcal{L}_D = -\log[1] - \log[1 - \mathbf{0.5}] = \mathbf{0.69}$

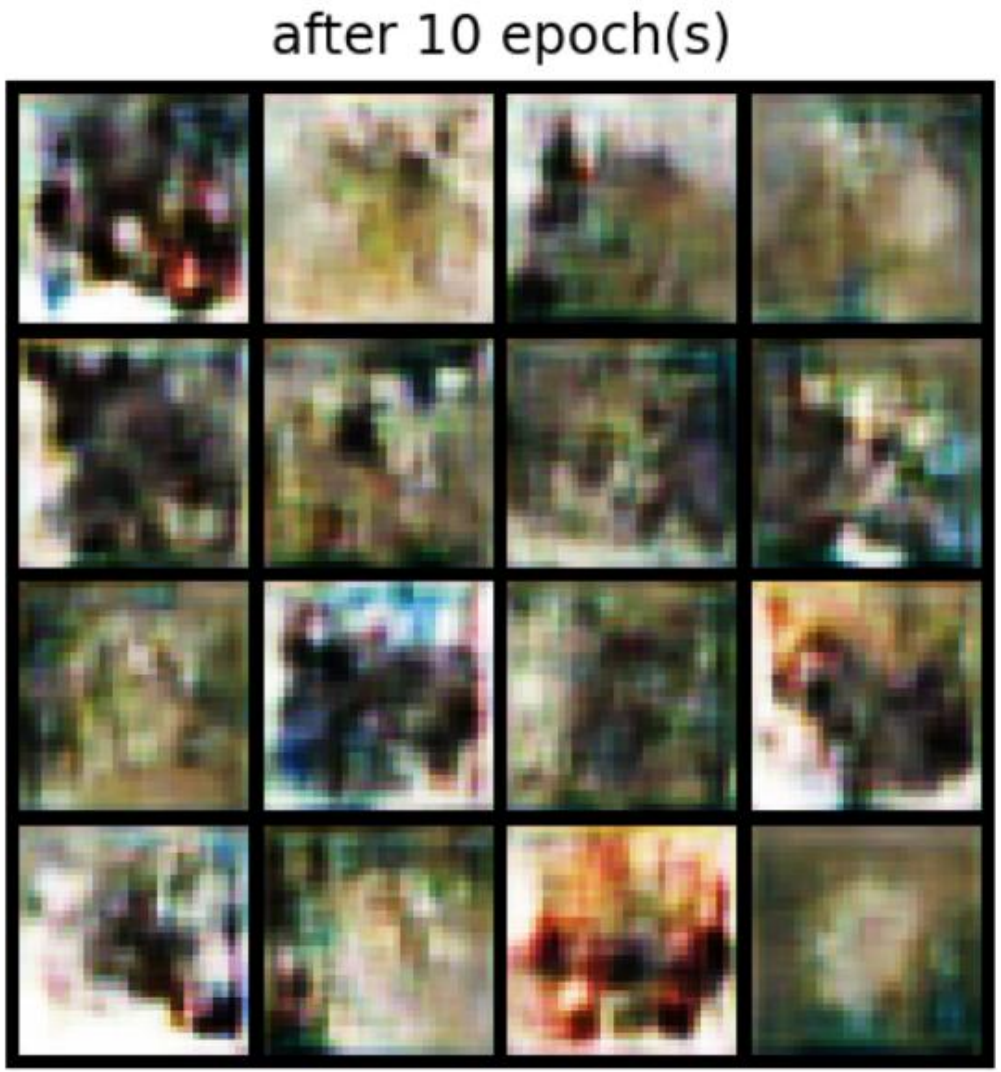
ideal case for *G*: $\mathcal{L}_G = \log[1 - \mathbf{0.5}] = \mathbf{-0.69}$
worst case for *G*: $\mathcal{L}_G = \log[1 - 0] = 0$

Convolutional Neural Networks:

Generative Adversarial Network (GAN)

check out `GAN architecture.ipynb`

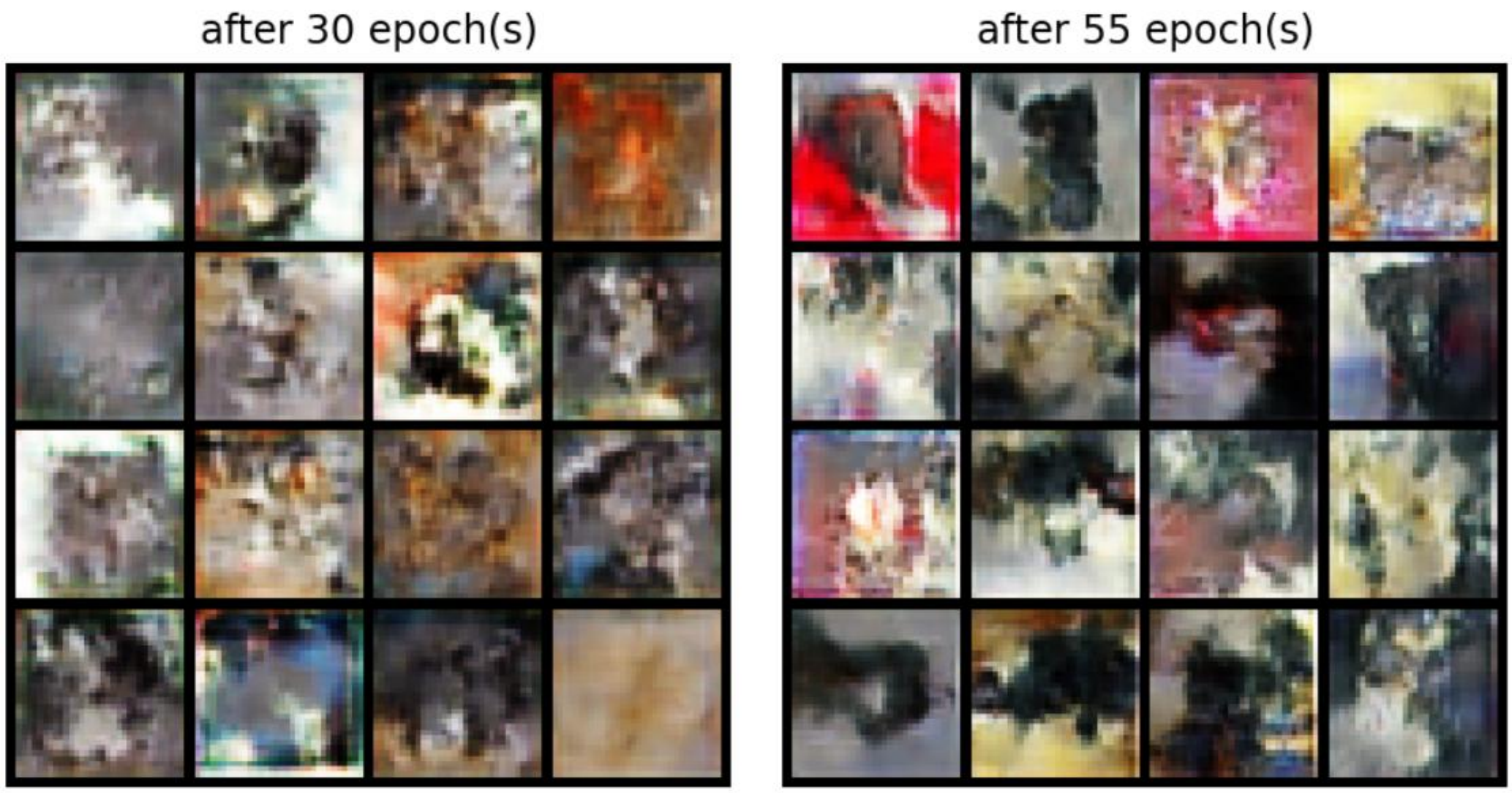




Convolutional Neural Networks:

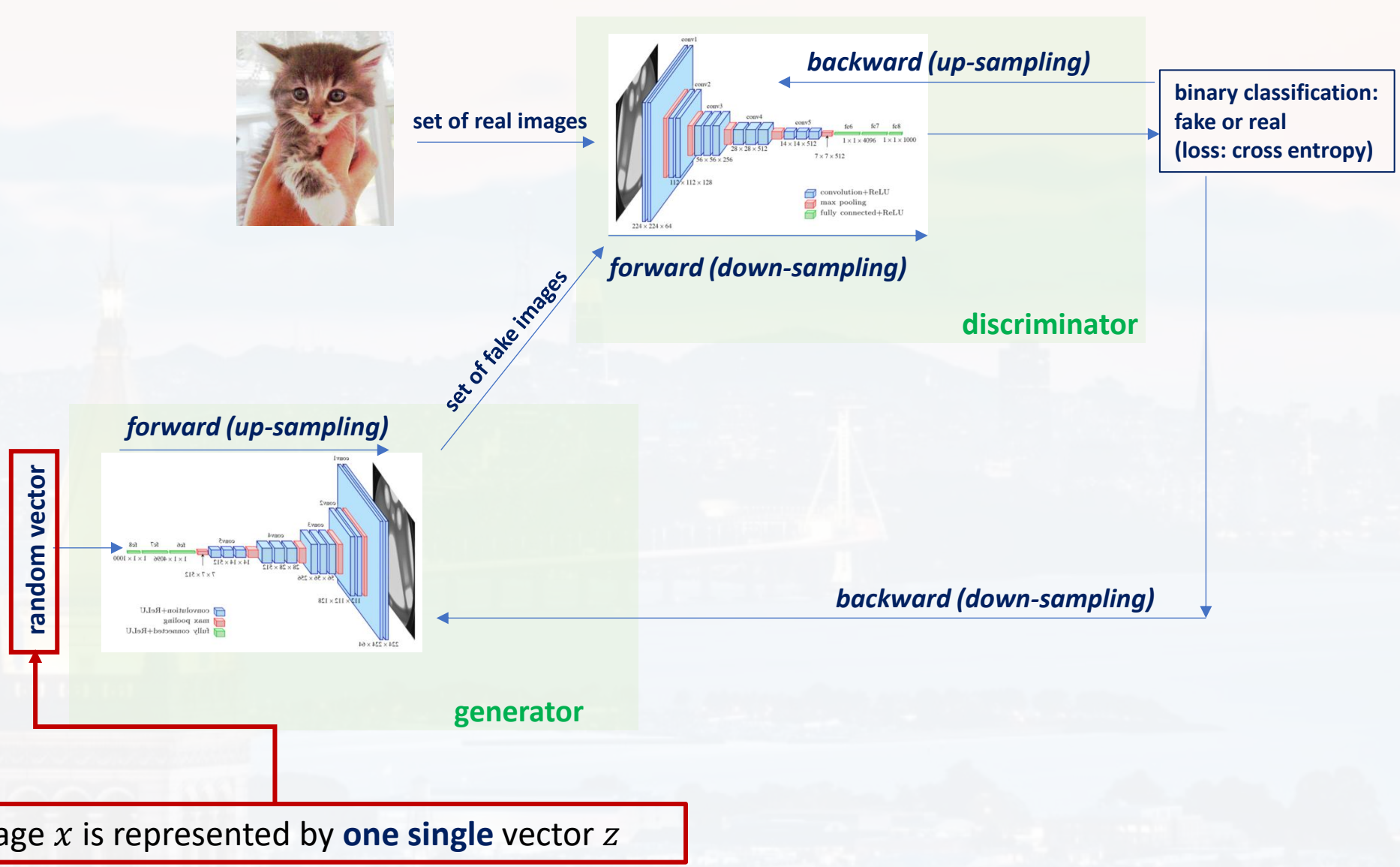
Generative Adversarial Network (GAN)

check out `GAN architecture.ipynb`



Convolutional Neural Networks:

GAN:

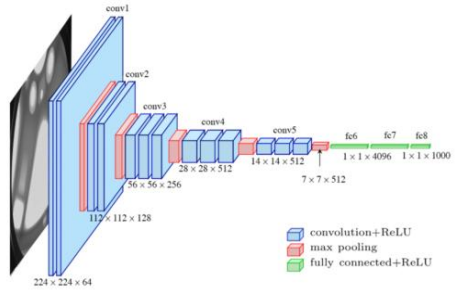


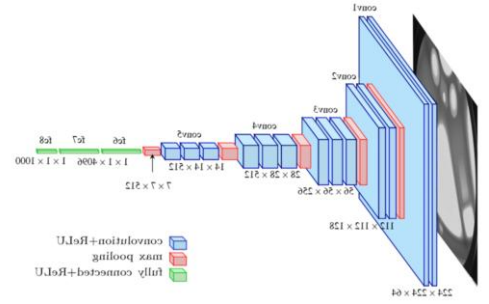
each image x is represented by **one single** vector z

Convolutional Neural Networks:

Variational AutoEncoder (VAE)

now:

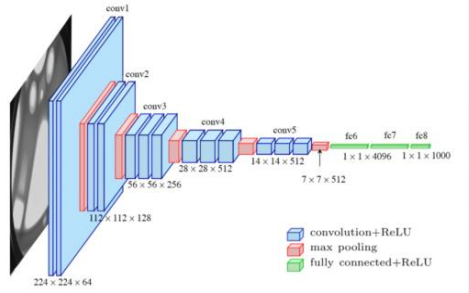


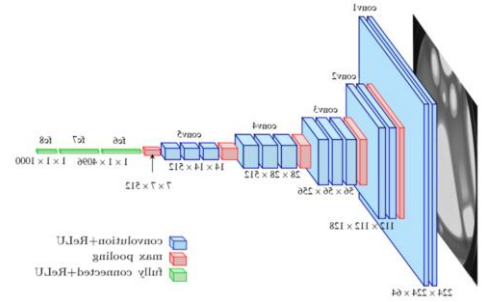


Convolutional Neural Networks:

Variational AutoEncoder (VAE)

now:





GAN: each image x is represented by **one single** vector z

VAE: each image x is represented by **a distribution** we sample from in order **to get z**

i: index over dimensions of z

$z_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$ (max entropy!)

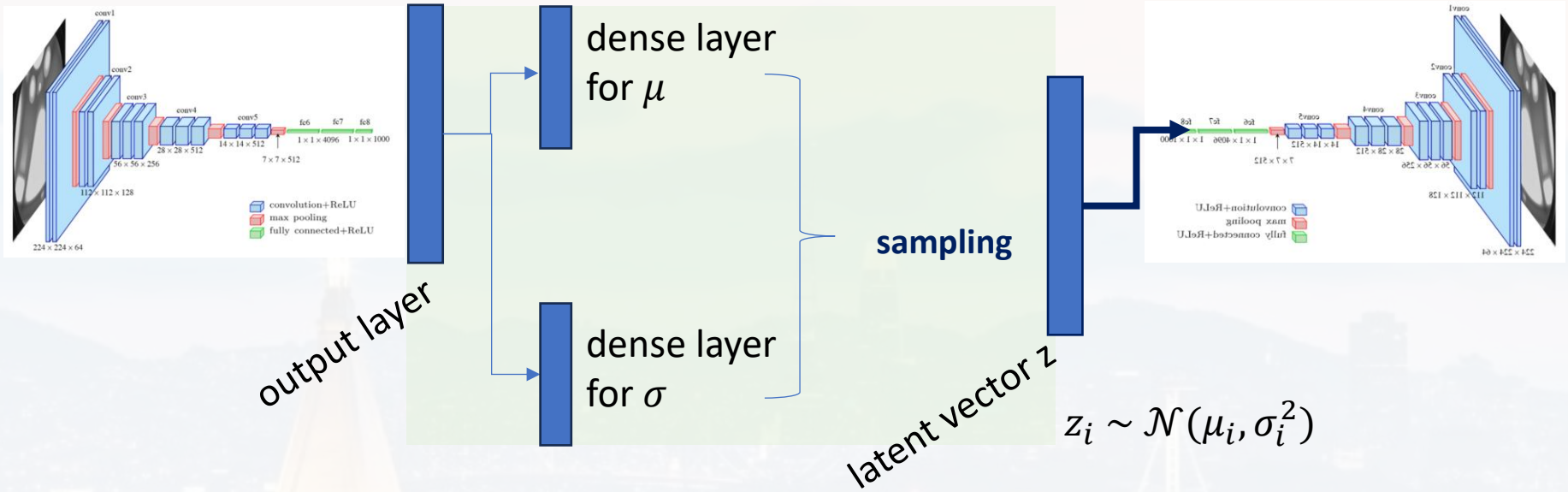
→ generate image from up-sampling as before

→ randomnes accounts for diverisity in generated data

Convolutional Neural Networks:

Variational AutoEncoder (VAE)

now:



$z_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$

example: output layer has shape (None, N)

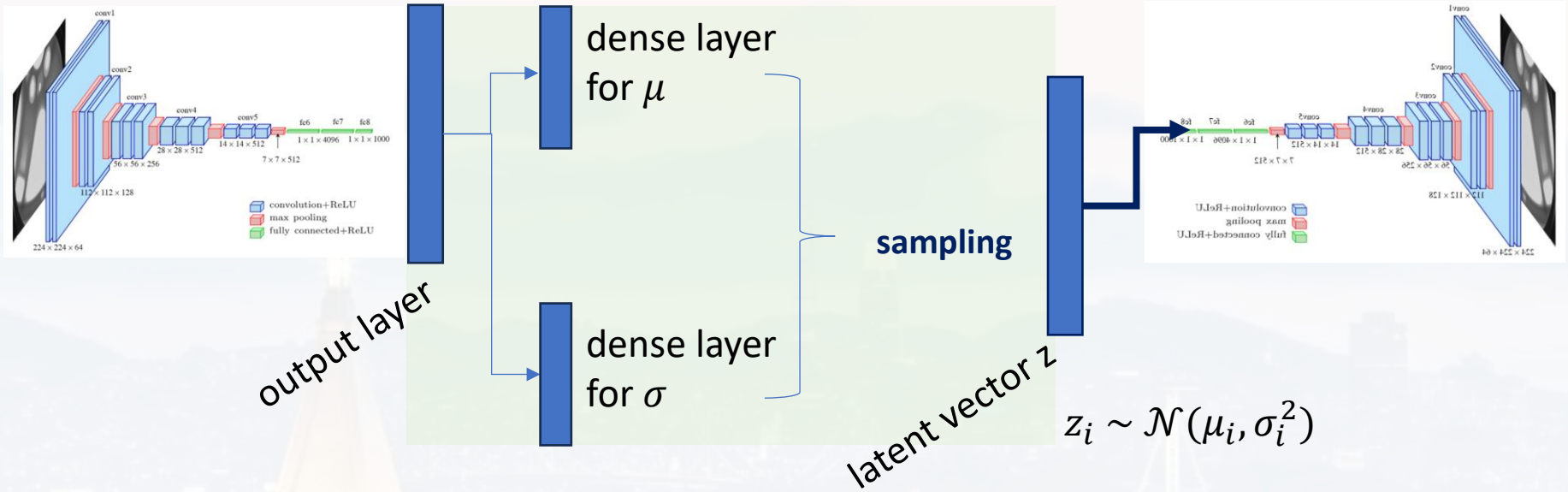
we want M dimensional encoding for z

→ weights in dense layer for σ and μ have shape (N, M)

→ latent vector has length M

Variational AutoEncoder (VAE)

now:



$z_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$

backprop:

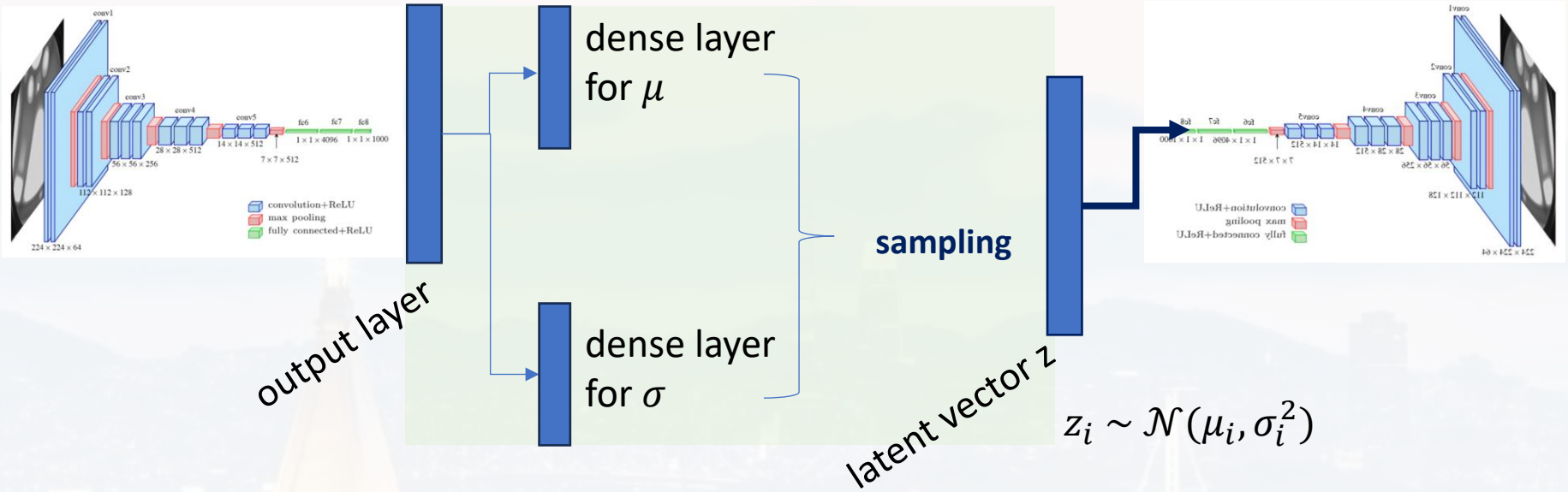
- μ_i and σ_i^2 have been generated deterministically from a dense layer
- we need to run backpropagation through μ_i and σ_i^2 in order to adjust the weights/biases
- in order to keep randomness, we actually sample via

$z_i = \mu_i + \sigma_i \cdot \epsilon$ where $\epsilon \sim \mathcal{N}(0,1)$ is not affected by backpropagation

Convolutional Neural Networks:

Variational AutoEncoder (VAE)

now:



loss: 1) **reconstruction loss L_R**, i. e. the difference between input image x and reconstructed image $\boldsymbol{p(x|z)}$. Note: L_R alone would **lead to overfitting**, exact image gets reproduced!

2) we need a **regularization term** that counteracts $\boldsymbol{L_R}$:

KL-divergence $\boldsymbol{L_{KL} = KL[q(z|x)|p(z)]}$, which increases if learned distribution $\boldsymbol{q(z|x)}$ gets too far from prior $\boldsymbol{p(z)} \sim \mathcal{N}(0,1)$

Variational AutoEncoder (VAE)

loss: 1) **reconstruction loss L_R**, i. e. the difference between input image x and reconstructed image $p(x|z)$. Note: L_R alone would **lead to overfitting**, exact image gets reproduced!

2) we need a **regularization term** that counteracts $\boldsymbol{L_R}$:

KL-divergence $\boldsymbol{L_{KL} = KL[q(z|x)|p(z)]}$, which increases if learned distribution $\boldsymbol{q(z|x)}$ gets too far from prior $\boldsymbol{p(z)}\sim\mathcal{N}(0,1)$

see here for more details

check out `VAR architecture.ipynb`

Convolutional Neural Networks:

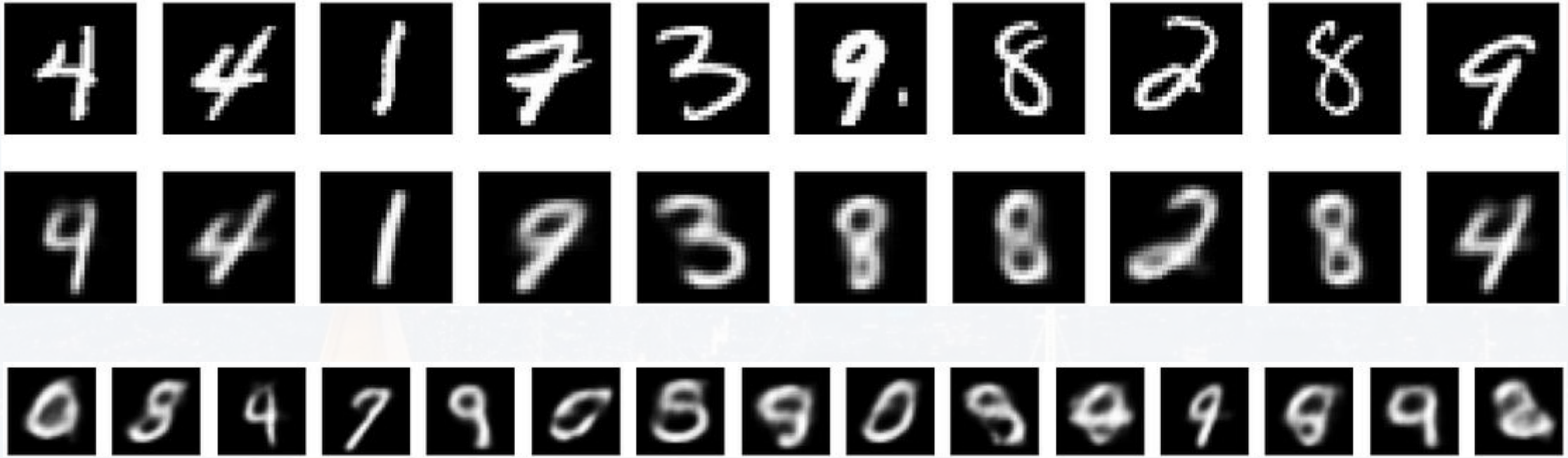
Variational AutoEncoder (VAE)

check out `VAR architecture.ipynb`

original

reconstruction

generated



Convolutional Neural Networks:

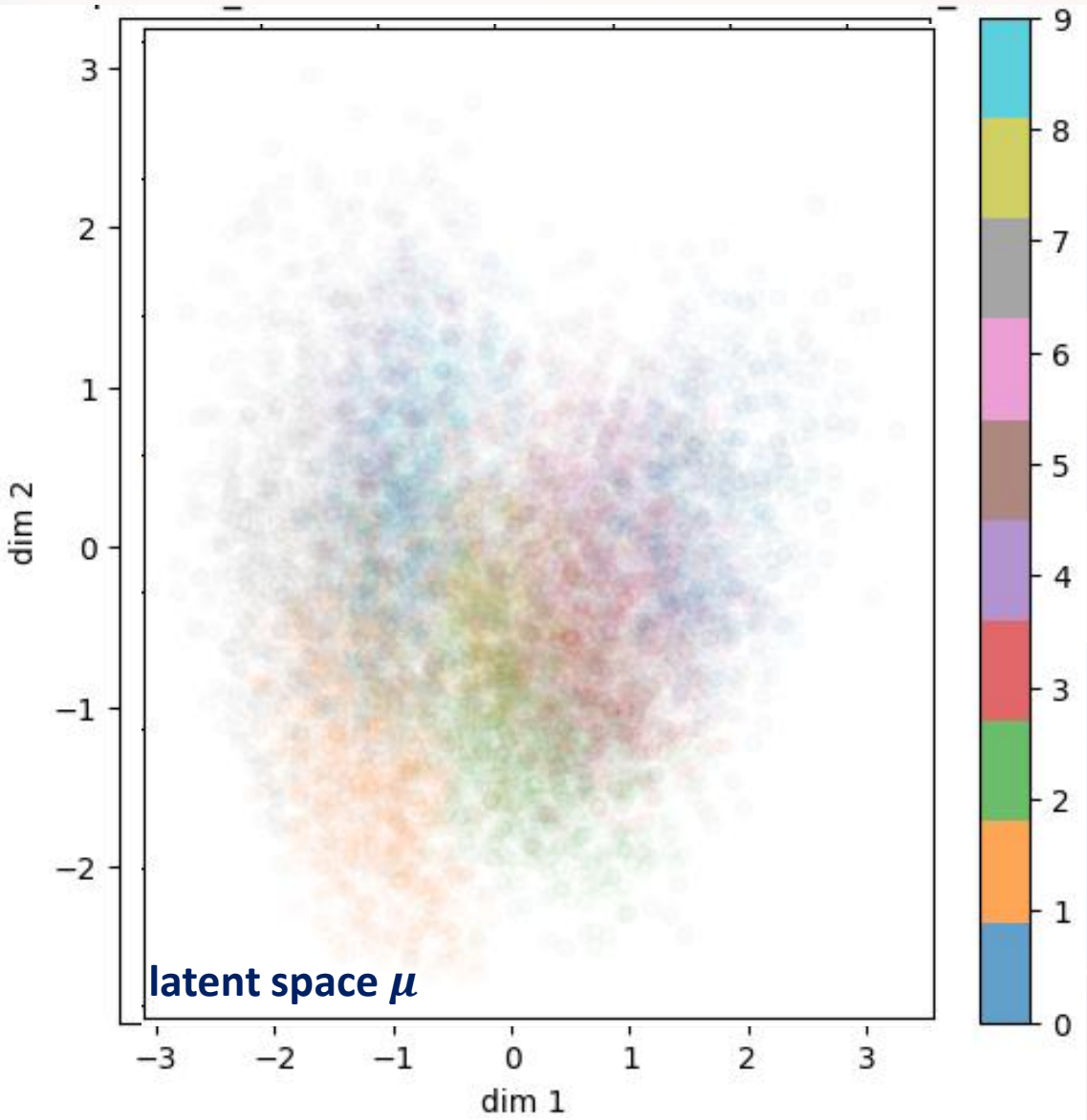
Variational AutoEncoder (VAE)

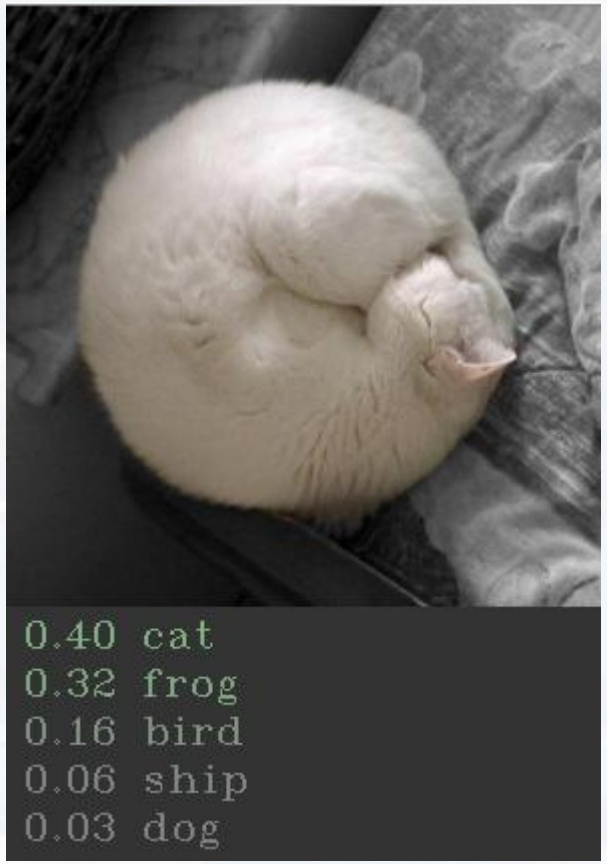
check out `VAR architecture.ipynb`

original

reconstruction

generated



Machine Learning Algorithms:

Thank you very much for your attention!